Network Traffic Anomaly Detection Using Sonification: A Working Model with Live Packet Capture and Five Design Improvements

Team members: Royce B Selvaraj (24BCB0078) · Chilukuri Charith Reddy (24BCT0093) · Vinay A (24BCB0131)
Guide: [Faculty name] **Department:** [Department] **Institution:** [College] **Date:** October 2026

Abstract

Security analysts watch network traffic mostly through text and visual displays. Sonification, the use of non-speech sound to convey data, offers a second channel that can be followed while doing other work. This project builds a working model of the packet-header sonification evaluated by Axon et al. (*Computers & Security*, 2019), in which trained listeners detected four attack conditions with an F-score of 0.92. Our model is a single web page that samples packet headers and plays each one as three notes: packet rate sets the tempo, packet size the note length, direction the stereo position, address and port the pitch register, protocol the instrument, and membership of a learned hotlist decides whether a note is consonant or dissonant. It accepts three traffic sources: a simulated network with four injectable attacks, a Wireshark capture file, and the live traffic of the host computer through a small capture helper. We then implement five optional fixes for weaknesses of the base design: separate signature sounds for floods and port scans, level scaling by flow density for FTP, a heartbeat that turns into a flatline tone during an outage, moving-average hotlists, and adaptive filtering. In instrumented runs every fix produced its signature sound within 2.6 s of the condition starting, and no attack signature sounded during 60 s of normal simulated traffic. On the live traffic of one laptop the 30-second hotlist went stale within a minute, which limits how far the base design carries over from an aggregated network link to a single host. No listener study was carried out; the results describe what the system plays, not how well people hear it.

Keywords: sonification, network-security monitoring, anomaly detection, packet headers, parameter mapping, Web Audio API, packet capture, DDoS, port scan.

1. Introduction

1.1 Background

Network attacks are detected by a combination of automated tools and people. Intrusion detection systems and statistical anomaly detectors handle the volume, but they can be inaccurate [15][16], so analysts in Security Operations Centres (SOCs) still verify alerts, catch what the tools miss and judge the overall state of the network. The displays that support this work are almost entirely visual or text based.

Sonification is "the use of non-speech audio to convey information" [13]. Sound can be followed in the background while the eyes are busy [14], and people are good at noticing when a familiar sound pattern changes [12]. Axon, Happa, Goldsmith and Creese [1] tested this idea for network security: 30

listeners heard sonified packet headers and were asked to detect and name attacks. Their study is the base paper for this project.

1.2 Problem statement

The base paper evaluates its sonification with pre-recorded audio made from synthetic datasets. The authors state that the system was not run in real time, and they report or leave open several weaknesses of the design: port scans mistaken for DDoS attacks, false alarms from ordinary FTP traffic, a connectivity loss that is heard only as silence, and listener fatigue that was not assessed. This project asks whether the design can be rebuilt as a working real-time model on ordinary hardware, whether it carries over to recorded and live traffic, and whether those weaknesses can be addressed without abandoning the original mapping.

1.3 Objectives

1. Reproduce the base paper's mapping from packet headers to sound as a working model that plays in real time in a web browser.
2. Model a network with the paper's four attack conditions so that each can be switched on and heard.
3. Play real traffic: Wireshark capture files and the live packets of the host computer.
4. Implement five fixes for weaknesses of the base design, each switchable, so that the original and improved designs can be compared directly.
5. Measure when each sound is produced and how often it is produced falsely, on simulated, recorded and live traffic.
6. Relate the observations to the findings of the base paper.

1.4 Scope

The model reads packet headers only (IPv4, IPv6, TCP, UDP and ICMP), never payloads. It covers the four conditions of the base paper (DDoS SYN flood, port scan, data exfiltration over FTP and connectivity loss) and one combined attack. Live capture covers the traffic of one Windows computer, not a mirrored switch port. The results are instrumented measurements of the system's output. A listener study, other attack types and other operating systems for the capture helper are outside the scope and listed as future work.

1.5 Report organisation

Section 2 gives background, Section 3 reviews the literature and states the research gap, Section 4 describes the design and methodology, Section 5 covers implementation, Section 6 presents results, Section 7 discusses limitations, and Section 8 concludes.

2. Background

2.1 Sonification and parameter mapping

Sonification designs fall into three families [17]. Event-based sonification plays a discrete sound for a defined event, such as an alert. Parameter-mapping sonification ties dimensions of the data to dimensions of sound, such as pitch, tempo or timbre, and plays continuously. Model-based sonification lets the user excite a model of the data and hear its response. The base paper uses parameter mapping, because packet headers are multi-dimensional and arrive continuously [1].

2.2 Packet headers

Every IP packet carries a source address, a destination address, a protocol number and a length [19]. TCP and UDP add source and destination ports, and TCP adds flags; a segment with SYN set and ACK clear is a request to open a connection [20]. These few fields are enough to describe who is talking to whom, over which service, in which direction and how much, without reading any content.

2.3 The four attack conditions

- **DDoS (TCP SYN flood).** Many sources send connection requests to one victim. In the headers: a surge of small incoming packets from addresses not seen before.
- **Port scan.** One or a few sources probe many ports of a host. In the headers: many different destination ports that are not normally used.
- **Data exfiltration over FTP.** An internal host uploads data to an unfamiliar external host. In the headers: sustained large outgoing packets on the FTP ports.
- **Connectivity loss.** Traffic stops. In the headers: nothing, which is the difficulty.

2.4 Hotlists, consonance and dissonance

A hotlist is the set of addresses or ports seen most often during a baseline period. The sonification plays a value that is in its hotlist as a note that fits the harmony (consonant) and any other value as a note that clashes with it (dissonant). In our model the consonant notes come from the C major pentatonic scale and the dissonant notes are C♯, F♯ and B, which form a minor second, a tritone and a major seventh against a constant low C.

2.5 Metrics

- **Packet rate** (packets per second, pps) and **note rate** (notes per second). Each sampled packet plays three notes.
- **Dissonant-note rate by register** (notes per second): how strongly the base design signals a condition, and in which pitch range.
- **Signature onset:** time from the start of a condition to its first signature sound.
- **Signature persistence:** time from the end of a condition to its last signature sound.
- **False signatures:** attack signatures (any signature other than the steady heartbeat) played during normal traffic.
- **Listener metrics of the base paper**, quoted for reference only: precision `tp/(tp+fp)`, recall `tp/(tp+fn)`, F-score `2PR/(P+R)` and mean detection time [1].

3. Literature review and base paper

3.1 Base paper

The base paper is L. Axon, J. Happa, M. Goldsmith and S. Creese, "Hearing attacks in network data: An effectiveness study," *Computers & Security*, vol. 83, 2019 [1]. It presents a parameter-mapping sonification of packet headers, designed with the authors' formal model for network sonification [2], and tests it with 30 computer science and cybersecurity researchers at the University of Oxford.

System. A sampled packet becomes a note event. Higher packet rate gives faster tempo and larger packets give longer notes. Incoming traffic is panned left and outgoing right. Source addresses play in a low pitch range, destination addresses in a middle range and destination ports in a high range. Values in a hotlist, computed from the first section of the dataset, play as consonant notes and all others as dissonant notes. HTTP plays on strings, other TCP on clarinet and FTP on piano. A Python program read the packets and sent Open Sound Control messages to the SuperCollider synthesis platform, and the output was recorded to audio files for the study.

Data. The datasets were synthetic: 25 desktop hosts talking to 40 remote hosts at an average of 4,500 packets per second, with four injected conditions (a SYN flood from 10 bots, a TCP connect scan from two scanners, FTP exfiltration to one external address, and a loss of connectivity).

Findings.

- After training, listeners detected attacks with precision 0.85, recall 1.0 and F-score 0.92. Before training, having heard only 30 s of normal traffic, they reached precision 0.61, recall 0.96 and F-score 0.75.
- Mean detection time after training was 2.43 s for the DDoS, 2.66 s for the port scan, 2.99 s for the exfiltration and 6.44 s for the connectivity loss. The authors' target was 3 s.
- Of 120 detected attacks, 114 were named correctly. All six errors were port scans named as DDoS.
- Of the 30 listeners, 24 recognised a combined attack and 16 of them named both parts.
- Musical experience had no significant effect on performance.

Weaknesses. The authors trace the scan and DDoS confusion to the pitch-range mapping: a scan from an unusual source also produces low-range dissonance, the main sign of a DDoS. Many false-positive clicks followed the piano notes of small, harmless FTP transfers. The connectivity loss was heard only as the sound thinning out, and some listeners took the silence for the end of the test. Listener fatigue was left to future work. We add one observation of our own: because the hotlists are fixed after the baseline, any later change in normal traffic stays dissonant indefinitely.

3.2 Related work

Sonification of network traffic. Gilfix and Couch [5] built Peep, which represents network events with natural sounds so that administrators can hear conditions such as excessive traffic. Ballora et al. [6] and Qi et al. [7] map traffic features to sound and report that denial-of-service and port-scanning activity become audible. Vickers et al. [8] sonify a network's self-organised criticality. These works describe systems; as [1] notes, they do not test with listeners how well attacks are heard. A survey by Axon et al. [4] summarises which traffic features and attack types earlier systems targeted.

User studies. Debashi and Vickers evaluated a flow-based sonification for situational awareness [9] and for recognising botnet behaviour [10]. Mancuso et al. [11] found that one sonified display did not improve performance when participants searched a capture for target packets. Axon et al. interviewed 21 security practitioners about where sonification could help in SOC work [3], and later studied sonification for the monitoring tasks of SOCs [18].

Sonification in general. The Sonification Handbook [12] covers design techniques and applications in fields such as medicine and astronomy. Hildebrandt et al. [14] show that a continuous sonification helps people monitor a process as a secondary task, which is the situation of a busy analyst.

Automated detection. Surveys by Chandola et al. [15] and García-Teodoro et al. [16] describe statistical and learning-based anomaly detection and its limits, including false alarms. Sonification is proposed as a complement to these tools, not a replacement [1].

3.3 Research gap and contribution

The base paper shows that people can hear attacks in pre-recorded sonifications of synthetic data. It does not run the system in real time, does not apply it to real traffic, and leaves its design weaknesses open. This project keeps the base paper's mapping and its four attack conditions and extends them as follows:

1. A real-time implementation in a web browser, with no software to install.
2. Three traffic sources behind the same mapping: a simulated network, Wireshark capture files, and live packets from the host computer.
3. Five switchable fixes, one for each weakness in Section 3.1, so that the original and improved designs can be heard side by side.
4. Instrumented measurements of what the system plays, including false signature sounds on real traffic.

4. System design and methodology

4.1 Overview

Every traffic source produces the same packet record: time, source address, destination address, service port, protocol, size, a SYN flag and direction. A sampler passes one packet in N to the mapper, which looks its addresses and port up in the hotlists and schedules three notes on the synthesiser. A scheduler runs every 25 ms and schedules notes 100 ms ahead, so timing does not depend on screen refresh. When any fix is switched on, a monitor computes anomaly scores from the sampled packets and drives the signature sounds. Figure 4.1 shows the structure.

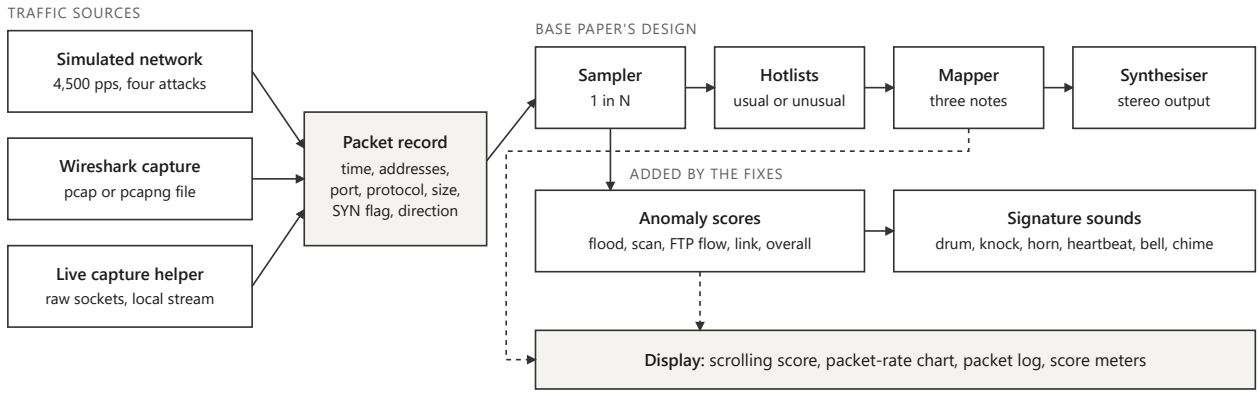


Figure 4.1. Structure of the model. The upper row reproduces the base paper's design. The lower row runs only when a fix is switched on.

4.2 Traffic sources

Simulated network. Twelve internal hosts exchange packets with fourteen external servers at about 4,500 packets per second, the rate used in [1]. Hosts and servers are drawn with Zipf weights, so a few are busy and most are quiet. The service ports are 443 (46%), 80 (22%), 22 (8%), 21 (7%), 993 (6%), 20 (6%) and 25 (5%); 56% of packets are incoming, and 1% involve a random external host that appears only once. An attack adds packets to this background (Table 4.3).

Our simulation is simpler than the base paper's in two ways, and we note them because they change the numbers. The base paper models each service as an on/off process and samples unusual packets with a higher probability. We generate a steady mix and sample uniformly, so we scale the attack rates up until an attack is a clear share of the sampled packets.

Wireshark capture file. The page reads pcap and pcapng files directly in the browser; nothing is uploaded. It decodes Ethernet (including VLAN tags), raw IP, loopback and Linux cooked captures, IPv4 and IPv6 with extension headers, and TCP, UDP and ICMP. Three interpretation rules apply. The service port is the lower of the two port numbers, so a reply sounds like the request it answers. A TCP segment that carries data on port 80 or 443 counts as HTTP and on port 20 or 21 as FTP, as in Wireshark's protocol column. A packet is outgoing if its source is a private-range address and its destination is not; when both ends are on the same side, the busiest address is taken to be the capturing host. The first 30 s of the capture (the first third of a shorter one) are the baseline that builds the hotlists.

Live traffic. A web page cannot read network packets, so a helper program, `live_capture.py`, does it. It opens a raw socket on the network interface that carries the default route, switches it to receive all packets, and for every packet sends the page a timestamp, the direction, the length and the header bytes (the IP header and the first 20 bytes of the transport header, not the payload). The stream is sent as server-sent events [22], one batch every 50 ms, to a page served on the local address 127.0.0.1 only. The page decodes the headers with the same code as for capture files. The first 30 s of live traffic are the baseline, and nothing plays until they have been learned.

4.3 Sonification mapping

Table 4.1: Mapping from packet header to sound

Header data	Sound	Implementation in this project
Packet rate	Tempo	One chord of three notes per sampled packet, at most 32 packets per second
Packet size	Note length	0.07 s for 40 bytes rising to 0.57 s for 1,500 bytes: `d = 0.07 + 0.5·((size − 40)/1460)^0.8`
Direction	Stereo position	Incoming panned left (−0.75), outgoing panned right (+0.75)
Field	Pitch register	Source address low (C2 to B3), destination address middle (C4 to C5), port high (C6 to C7)
Hotlist membership	Consonance	In the hotlist: a note of the C major pentatonic scale, chosen by the value's rank. Not in the hotlist: C♯, F♯ or B, chosen by a hash of the value, so the same unusual value always sounds the same
Protocol	Instrument	HTTP on strings (two detuned sawtooth waves), other TCP on clarinet (filtered square wave), FTP on piano (triangle wave with two partials and a decay), anything else on strings
Traffic present	Drone	A soft low C that gives dissonant notes something to clash with and fades when traffic stops (our addition)

The first six rows follow Table 1 of the base paper. Two details differ: the base paper plays an unusual value at a random dissonant frequency and allows notes of up to 1 s.

4.4 Hotlists and sampling

The hotlists hold the 22 most frequent addresses and the 7 most frequent ports of the baseline. For simulated traffic the baseline is 135,000 packets (30 s at 4,500 pps) and the sampler passes 1 packet in 600, which gives about 7.5 packets, or 22 notes, per second. For a capture file or live traffic the sampling ratio is set from the baseline rate so that normal traffic plays at the same 7.5 packets per second.

4.5 Anomaly scores

The base design has no detector: the listener is the detector. The fixes need to know when to act, so a monitor computes five scores between 0 and 1 from the packets sampled in the last 2 s. `clamp` limits a value to the range 0 to 1.

- **Flood:** `F = clamp((S − 5)/7)`, where S is the number of different unusual sources that sent an incoming connection request (SYN).
- **Scan:** `P = clamp((U − 1)/5)`, where U is the number of different unusual ports.
- **FTP flow density:** each source-to-destination FTP flow keeps a counter `n ← min(8, n·e^(−Δt/3) + 1)` per sampled packet, and `T = clamp((n_max − 2)/4)`.
- **Link:** for simulated traffic and capture files, L is the packet rate of the last second divided by the baseline rate. For live traffic, `L = clamp(2 − 2g/q)`, where g is the time since the last packet and q is twice the longest silence in the baseline, and at least 2 s.
- **Overall:** `A = max(F, P, T, clamp((0.6 − L)/0.35), X)`, where `X = clamp((s − s₀ − 0.2)/0.2)`, s is the share of sampled packets that carry any unusual value and s_o is the same share in the

baseline. X is not used for live traffic (Section 6.5).

4.6 The five fixes

Table 4.2: Weaknesses of the base design and the fixes implemented

Issue in the base paper	Fix	Implementation	Signature sound
DDoS and port scan confused: six scans named as DDoS	Separate timbres	While $F > 0.3$ a drum plays at 3 to 5 hits per second. Each unusual port not heard in the last 2 s plays a knock in place of its high note.	Drum (flood), knocking (scan)
FTP false positives: every FTP packet plays the same distinct piano note	Level follows flow density	An FTP note plays at 30% level for a lone packet and up to 170% for a dense flow: `clamp(0.3 + 0.28(n − 1), 0.3, 1.7)`. While $T >$ 0.25 a horn sounds every 1.1 s.	Car horn
Connectivity loss is silence: slowest to detect at 6.44 s	Heartbeat carrier tone	A heartbeat plays once a second while $L \geq 0.3$. When $L < 0.3$ it stops and an 880 Hz tone sounds until traffic returns.	Heartbeat, then flatline tone
Static hotlists: fixed after the baseline	Moving-average hotlists	Each value has a weight `w ← w·e^(−Δt/45) + 1` per sampled packet. A new value joins the hotlist once it has been seen for 30 s with no gap over 10 s. Members keep a 25% margin so the list does not flicker, and a value whose weight falls below 0.05 is forgotten.	Rising chime when a new address is learned
Auditory fatigue: not assessed	Adaptive filtering and muting	Notes pass through a 1.1 kHz low-pass filter at half level. When $A > 0.4$ a bell rings, the filter opens to 16 kHz and the level rises by 25%. It closes after A has stayed at or below 0.25 for 1.5 s.	Bell

All five are off by default, and with all five off the page plays the base paper's design unchanged. The signature sounds are event-based sonification added on top of the parameter mapping: the notes still play, and the signatures name what the monitor has found.

4.7 Test scenarios

Table 4.3: Conditions injected into the simulated network

Condition	Added traffic	Packets
DDoS SYN flood	+9,000 pps	SYN packets of 40 to 60 bytes from random sources to one internal host, port 80
Port scan	+2,600 pps	SYN packets of 44 bytes from one external source to one internal host, ports 1 to 1,024 in rising order
Data exfiltration	+1,600 pps	FTP data packets of 1,380 to 1,500 bytes from one internal host to one unknown external host
Connectivity loss	to 0 pps	All traffic fades out over 3.2 s
Combined attack	+10,600 pps	The flood and the exfiltration together
Traffic shift (harmless)	none	Three busy external servers move to new addresses

An attack ramps up over 1.2 s and down over 0.8 s. Two further sources test the model beyond the simulation.

- **Sample capture.** A synthetic pcapng file made for the project: 36,654 packets over 118 s at about 120 pps, with a SYN flood from 40 to 50 s (about 1,860 pps), a port scan from 62 to 70 s, an FTP transfer from 80 to 92 s and an outage from 100 to 108 s.
- **Live test feed.** A test program that hands synthetic packets to the capture helper in place of the raw sockets, so that the whole live path can be tested repeatably and without administrator rights. It produces bursty web traffic at about 64 pps and, on request, a page load from several hundred new servers, an outgoing port scan, an incoming SYN flood, an FTP upload, silence, and steady use of one new server.

4.8 Parameters

Table 4.4: Parameters of the model

Parameter	Value
Sampling ratio	1 in 600 (simulated); baseline rate ÷ 7.5 (capture file and live)
Hotlist size	22 addresses, 7 ports
Baseline	30 s
Scheduler	Every 25 ms, notes scheduled 100 ms ahead
Maximum note rate	32 packets (96 notes) per second
Analysis window for the scores	2 s
Moving-average hotlist	Time constant 45 s, probation 30 s, largest gap 10 s
FTP flow counter	Time constant 3 s, capped at 8
Live stream	Batches every 50 ms; playback runs 0.3 s behind capture

4.9 Measurement method

We measured what the page plays by instrumenting its audio output in the browser. A test script logs every oscillator the page starts, with the time at which it will sound. Signature sounds are recognised by the routine that generates them. Instrument notes are recognised by waveform and pitch, which gives the instrument, the register and whether the note is consonant or dissonant. The script switches conditions on and off at fixed times and records counts and times, which are accurate to about 0.1 s. No human listeners took part.

4.10 Experiment plan

1. Simulated traffic, all fixes off: 30 s of normal traffic, then each condition for 8 s.
2. Simulated traffic, all fixes on: 60 s of normal traffic, each condition for 8 s followed by 8 s of recovery, then the traffic shift for 75 s.
3. The sample capture from start to end, all fixes on.
4. The live source driven by the test feed, all fixes on: 30 s of learning, 40 s of normal traffic, then each condition for 8 s.
5. The live source on the real traffic of one laptop: the capture path, the baseline, and false signatures during ordinary use.

5. Implementation

The model is one self-contained web page, `Network-Sonifier.html` (about 1,200 lines of HTML, CSS and JavaScript), and one helper script, `live_capture.py` (about 140 lines of Python using only the standard library). Figure 5.1 shows it running. Its components are:

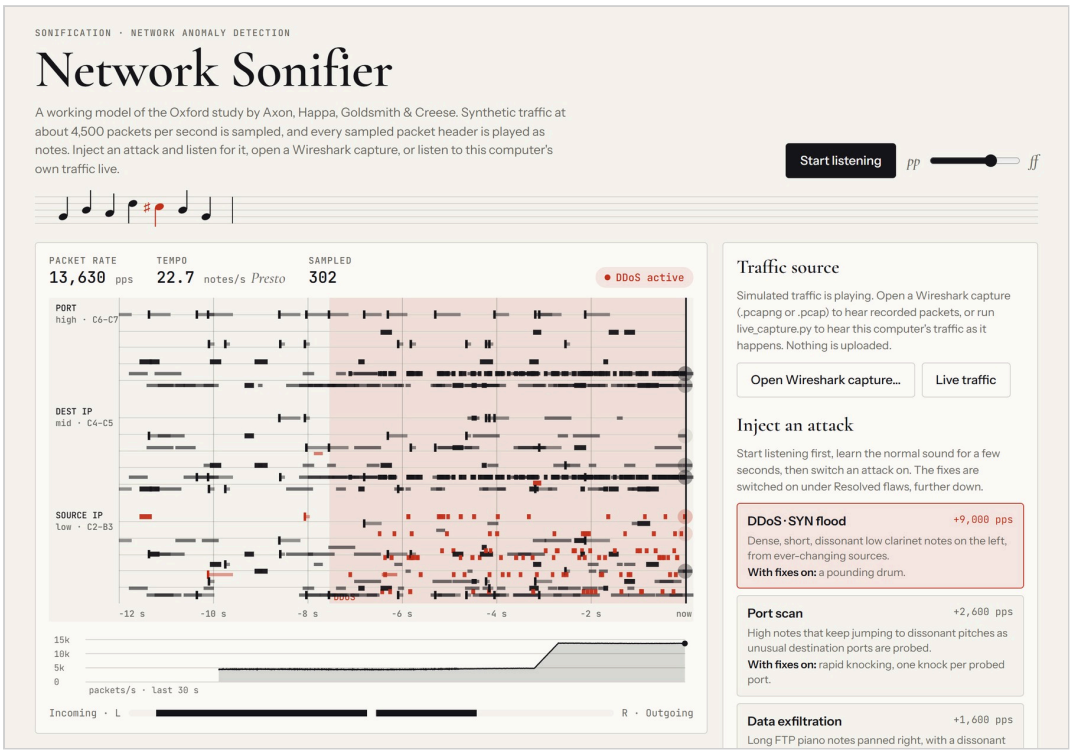


Figure 5.1. The model playing simulated traffic with a SYN flood injected and all fixes off. The lowest staff (source address) fills with red dissonant notes as new sources arrive, and the packet rate rises from 4,500 to about 13,600 pps.

- **Traffic model:** generates normal and attack packets for the simulated network (Section 4.2 and Table 4.3).
- **Capture parser:** reads pcap and pcapng files with a `DataView` and decodes link, IP and transport headers.
- **Live source:** receives the helper's stream with an `EventSource`, decodes it with the capture parser's header decoder and aligns the helper's clock with the page's.
- **Hotlists and pitch mapper:** Sections 4.3 and 4.4.
- **Synthesiser:** built on the Web Audio API [21] from oscillators, filters, a stereo panner and a compressor. No audio files are used.
- **Engine:** the 25 ms scheduler that samples packets and schedules notes.
- **Resolved-flaws module:** the anomaly scores of Section 4.5 and the five fixes of Table 4.2.
- **Display:** a scrolling score with one staff per register and unusual values in red, a packet-rate chart, a left and right level meter, a log of sampled packets, the hotlists and the score meters.
- **Controls:** traffic source, attack buttons, sampling ratio, hotlist size, share of rare hosts, and one switch per fix.
- **Capture helper:** one thread per IP version reads a raw socket in receive-all mode [23] and restarts when the network changes; a small HTTP server streams the headers to the page.

Windows allows this only to an administrator, so the script asks for elevation when started.

Tool disclosure: The code of the model, the test scripts and the instrumented runs reported in Section 6 were produced with the assistance of an AI coding tool (Claude Code, by Anthropic), and this report was drafted with the same tool from those runs. [Add what the team reviewed, tested and changed, in line with your department's policy.]

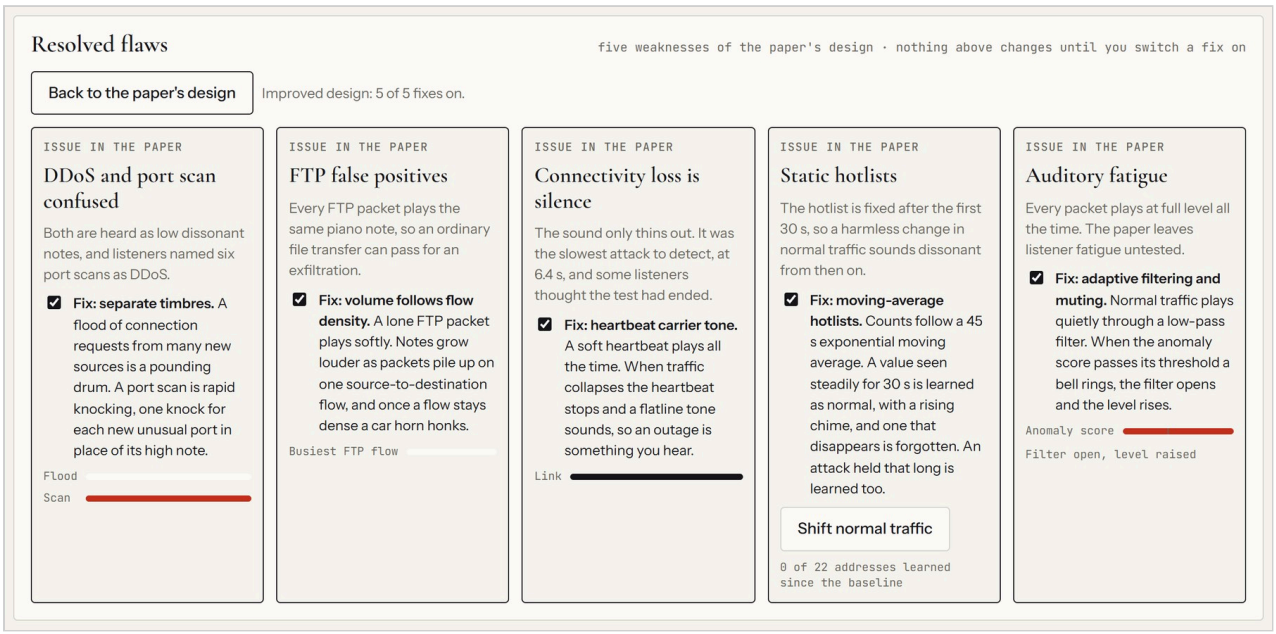


Figure 5.2. The Resolved flaws section with all five fixes on during a port scan. The scan score is full, the flood score is empty, and the adaptive filter has opened.



Figure 5.3. The live source after the 30 s baseline has been learned, here driven by the test feed of Section 4.7. The attack buttons are disabled because a live source plays captured packets only.

6. Results and analysis

All values in this section come from our own instrumented runs on 11 and 12 October 2026, on a Windows 11 laptop in a Chromium-based browser. Tables 6.1 to 6.4 were measured on the final code. Values quoted from the base paper are marked [1].

6.1 Expected behaviour (hypotheses)

- **H1.** With all fixes off, each condition changes the sound as [1] describes: a flood adds dissonant low clarinet notes, a scan adds dissonant high notes, an exfiltration adds piano notes and a dissonant middle note, and a connectivity loss thins the sound to silence.
- **H2.** A port scan also raises dissonance in the low register, which is the cause [1] gives for scans being named as DDoS.
- **H3.** With all fixes on, each condition produces its own signature sound within 3 s, the detection-time target of [1], and no attack signature sounds during normal traffic.
- **H4.** The moving-average hotlist learns a harmless change in normal traffic in about 30 s.
- **H5.** The design carries over from the simulated network to a capture file and to live traffic without retuning.

6.2 Results tables

Table 6.1: Base design (all fixes off), simulated traffic. Notes per second in the first 8 s of each condition

Condition	All notes	Dissonant, low	Dissonant, mid	Dissonant, high	Strings	Clarinet	Piano
Normal (30 s)	21.4	0.27	0.13	0	14.1	4.2	3.1
DDoS SYN flood	64.5	13.62	0.62	0	16.1	45.3	3.0
Port scan	43.5	4.74	0.12	4.25	21.4	17.2	4.9
Data exfiltration	36.3	0.50	2.25	0	18.7	4.1	13.5
Connectivity loss	6.0	0.12	0	0	3.7	1.1	1.1
Combined attack	73.7	14.85	2.50	0	17.2	47.2	9.4

No signature sound (drum, knock, horn, heartbeat, flatline, bell or chime) was played in any row of Table 6.1.

Table 6.2: Improved design (all fixes on), simulated traffic. Times are from the start of the condition

Condition	Signature	First heard	After the condition ended	Other signatures
Normal (60 s)	60 heartbeats	n/a	n/a	None
DDoS SYN flood (8 s)	35 drum hits	1.1 s	Last hit at 1.9 s	Bell at 1.0 s
Port scan (8 s)	35 knocks	0.7 s	Last knock at 0.4 s	Bell at 0.7 s
Data exfiltration (8 s)	6 horn blasts	1.5 s	Last blast at 3.4 s	Bell at 1.7 s
Connectivity loss (8 s)	Heartbeat stops after 1.9 s; flatline tone	2.6 s	Tone ends at 0.9 s; heartbeat back at 1.0 s	Bell at 2.1 s
Combined attack (8 s)	31 drum hits and 6 horn blasts	1.8 s, 2.4 s	Both stop by 2.1 s	Bell at 0.8 s
Traffic shift (75 s)	3 chimes, one per moved server	30.4 s	n/a	Bells at 1.7 s and 57.1 s

The heartbeat continued at one beat per second through every row except the connectivity loss. No drum sounded during the scan and no knock during the flood.

Table 6.3: Sample capture, all fixes on. Times are capture time

Event in the capture	Signature	Sounded from, to	Bell
SYN flood, 40 to 50 s	59 drum hits	40.2 s to 51.8 s	40.0 s
Port scan, 62 to 70 s	111 knocks	62.1 s to 69.6 s	62.0 s
FTP transfer, 80 to 92 s	14 horn blasts and 1 knock	80.0 s to 94.3 s	80.1 s
Outage, 100 to 108 s	Flatline tone; no heartbeat	101.0 s to 108.5 s	100.9 s
Rest of the capture	111 heartbeats, no other signature	0.9 s to 100.9 s, then 108.6 s to 117.6 s	0.3 s (see Section 7)

Table 6.4: Live source driven by the test feed, all fixes on. Baseline 1,907 packets in 30 s, sampling 1 in 8, hotlists of 15 addresses and 4 ports

Condition	Signature	First heard	After the condition ended
Normal (40 s)	40 heartbeats, nothing else	n/a	n/a
Page load from new servers (8 s)	None: no drum, no bell	n/a	n/a
Outgoing port scan (8 s)	86 knocks; bell	0.5 s	Last knock at 0.3 s
Incoming SYN flood (8 s)	36 drum hits; bell; 1 knock	0.9 s	Last hit at 2.1 s
FTP upload (8 s)	7 horn blasts; bell; 1 knock	0.7 s	Last blast at 2.5 s
Silence (8 s)	Heartbeat stops after 2.0 s; flatline tone; bell	2.4 s	Tone ends and heartbeat returns at 0.5 s
Steady use of one new server (42 s)	1 chime; the server joins the hotlist	31.4 s	n/a

Table 6.5: Live source on the real traffic of one laptop (IPv4, 11 October 2026)

Measurement	Result
Packets reaching the page in 8 s	522 (273 incoming, 249 outgoing; 325 UDP, 196 TCP, 1 ICMP)
Delay from capture to the page	Median 24 ms, maximum 51 ms
Baselines observed (30 s each)	589 packets with 27 addresses; 5,501 packets with 30 addresses; 11 ports in both
Baseline packets with a value outside the hotlists	1% and 0%
Sampled packets with a value outside the hotlists, per 2 s window after the baseline	Run of 80 s: median 33%, 90th percentile 97%. Run of 150 s: median 0%, 90th percentile 18%, maximum 100%
Longest silence	0.76 s in 150 s, against a limit of 2 s
Scan and flood scores, 150 s of ordinary use	Never above the threshold; 4 knocks
All fixes on, 60 s of ordinary use	60 heartbeats, 1 knock; no drum, horn, bell, chime or flatline tone

The last row was measured in the page. The other rows were measured by scripts that read the helper's stream and apply the page's rules. These runs were made on 11 October, before the second and fourth corrections in Table 6.6; neither of those changes the live score.

6.3 Figures

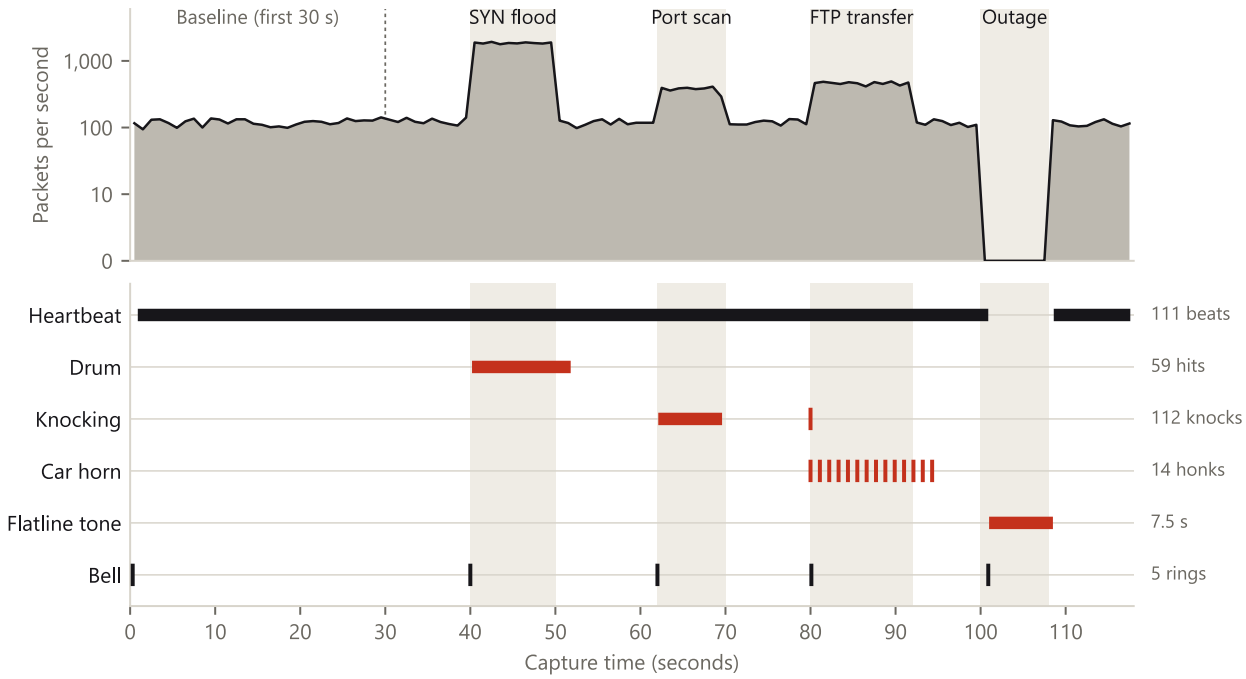


Figure 6.1. The sample capture played with all fixes on. Top: packets per second in the capture (logarithmic scale). Bottom: when each signature sound was played. Shaded bands mark the four events in the capture. Each event triggers its own signature and one bell. The bell at 0.3 s is a start-up artefact (Section 7).

6.4 Analysis

The base design (H1, H2). H1 holds. Under the flood the note rate triples, from 21.4 to 64.5 notes per second, dissonant low notes rise from 0.27 to 13.6 per second and clarinet notes from 4.2 to 45.3. The scan adds 4.25 dissonant high notes per second where normal traffic has none. The exfiltration raises piano notes from 3.1 to 13.5 per second and dissonant middle notes from 0.13 to 2.25. The connectivity loss cuts the note rate to 6.0 over the 8 s window, which includes the fade. H2 also holds: the scan raises dissonant low notes to 4.74 per second, about eighteen times the normal rate, on the same instrument as the flood. A scan and a flood therefore share their most prominent cue and differ mainly in how much of it there is and in the high notes on top, which is the confusion listeners reported in [1].

The fixes (H3). H3 holds for the simulated network, the capture file and the test feed. Every condition produced its own signature within 2.6 s of beginning, and neither 60 s of normal simulated traffic nor 40 s of normal test-feed traffic produced an attack signature. The signatures do not cross over: the flood played the drum and no knock, and the scan played knocks and no drum. In the test feed the flood and the FTP upload each added a single knock, because their ports (80 and 20) were not in that feed's port hotlist; this is the knock doing its job of marking an unusual port. For the connectivity loss the flatline tone begins 2.6 s after the loss starts, and the heartbeat has already stopped at 1.9 s. Listeners in [1] needed 6.44 s on average with silence as the only cue. The two numbers are not directly comparable, since one is when a sound starts and the other is when a person reacts, but the fix replaces an absence with a sound, which is what participants in [1] asked for.

Persistence. The drum stops about 2 s after a flood ends and the knocking within 0.5 s. The horn lasts longest, up to 3.4 s after a transfer ends, because the flow counter decays with a 3 s time constant.

Moving-average hotlist (H4). H4 holds. After the traffic shift the three moved servers were learned at 30.4, 31.5 and 32.5 s, each with a chime, and in the test feed a new server in steady use was learned at 31.4 s. Addresses seen only briefly, such as the servers of a single page load, were not learned. The cost is stated in the design: an attack sustained for more than 30 s is learned as normal too.

Corrections made after measurement. The first version of the fixes passed informal listening but not measurement. Table 6.6 lists what the instrumented runs exposed and how each rule was changed. All other tables in this section were measured after these changes.

Table 6.6: False signatures found by measurement, and the corrections

Measured problem	Cause	Change	After
Drum during a page load from new servers: 68 hits in 14 s (test feed)	The flood score counted every incoming packet from an unusual source	Count only connection requests (SYN)	0 hits
Bell during normal simulated traffic: 6 in 60 s	A fixed threshold on the share of unusual packets, although about 9% of normal packets are unusual and a 2 s window holds only about 15 packets	Measure the share against its level in the baseline	0 in 60 s
Bell during ordinary use of the laptop: 3 in 20 s (real live traffic)	A single computer's traffic moves on to addresses its 30 s baseline never saw	Leave the share of unusual packets out of the live score	0 in 60 s
Chime with nothing new: 1 in 60 s of normal simulated traffic	A baseline address entering the list from just outside it counted as learned	Chime only for addresses the baseline never saw	0 in 60 s
Chimes for addresses seen only briefly: 2 (test feed)	Probation measured time since first sight only	Require steady presence, with no gap over 10 s	0 chimes
Horn still sounding 5.5 s after a dense transfer ended (test feed)	The flow counter had no upper limit	Cap the counter at 8	2.5 to 3.4 s

6.5 Effect of the traffic source

H5 holds for the capture file and only partly for live traffic. The sample capture needed no retuning: with the sampling ratio set from its baseline, all four events produced the same signatures as in the simulation (Table 6.3 and Figure 6.1).

Live traffic from one laptop behaves differently from the aggregated link that [1] models, in three ways that the measurements in Table 6.5 make visible.

- **The hotlist goes stale quickly.** During the baseline almost every packet matched the hotlists (0% to 1% did not). In the minutes that followed, the median 2 s window in one run had 33% of its packets carrying a value the baseline had never seen, and the busiest windows more than 90%. A laptop keeps contacting new servers. The static hotlist weakness of the base design, which takes a deliberate traffic shift to show in simulation, appears on its own within a minute on a real host. For this reason the live score ignores the share of unusual packets.
- **Rate is bursty.** A host's packet rate swings between bursts and near silence, so a rule that compares the current rate with the baseline average would report outages constantly. The live source therefore judges an outage by silence: the longest gap in 150 s of ordinary use was 0.76 s, well inside the 2 s limit.

- **Replies look like a flood.** Loading a web page brings replies from many servers the host has never contacted before. Counting only connection requests separates this from a real SYN flood.

With these three changes, 60 s of ordinary use with all fixes on produced a steady heartbeat, one knock and no other signature.

7. Discussion and limitations

- **No listener study.** We counted the sounds the system plays. Whether people detect attacks faster or name them more accurately with the fixes, which is what [1] measured, has not been tested.
- **Detectors behind the signatures.** The signature sounds depend on the anomaly scores, so they inherit the blind spots of those scores. A UDP flood does not raise the flood score, and an attack built to resemble normal traffic raises none of them. The parameter-mapped notes still play in those cases, as in the base design.
- **Hand-set thresholds and short runs.** The thresholds were chosen by trial and corrected by measurement (Table 6.6). The false-signature figures come from runs of 40 to 150 s and are not long-term rates.
- **Simplified simulation.** Traffic is a steady mix with uniform sampling and enlarged attack rates, not the on/off service model and weighted sampling of [1].
- **Synthesised instruments.** Strings, clarinet and piano are approximated with basic waveforms, so their timbres are less distinct than sampled instruments.
- **Learning attacks.** The moving-average hotlist learns anything that persists for 30 s, including an attack.
- **Fatigue not evaluated.** The adaptive filter lowers the level of normal traffic, by roughly 3 dB in the low register and 10 dB in the high register in our measurement, but whether this reduces fatigue needs listeners and long sessions.
- **Live capture.** The helper captures one computer's traffic, not a network link; it runs on Windows only and needs administrator rights. The network used for testing had no IPv6, so IPv6 capture was exercised only with the test feed. A real outage and real attack traffic were not produced on the live network.
- **Start of a capture.** One bell rings 0.3 s after playback of a capture begins, because the one-second rate window is still empty and looks like an outage.
- **Headers only.** Attacks visible only in packet contents are out of reach, as in [1].

8. Conclusion and future work

Conclusion. The base paper's sonification can be rebuilt as a real-time model that runs in a web browser without installed software (objectives 1 and 2). With all fixes off it reproduces the cues the paper describes, and it also reproduces the cause of the paper's best-known weakness: a port scan and a flood share their low-register dissonance (Table 6.1). The model plays Wireshark captures and the live traffic of its host, with a median delay of 24 ms from capture to delivery (objective 3). The five fixes each produce a distinct signature within 2.6 s of a condition starting and no attack signature during

normal simulated traffic (objectives 4 and 5). The step from simulation to a live host was the instructive one: the 30-second hotlist that works on an aggregated link went stale within a minute on a laptop, and three detection rules had to change before ordinary use stopped triggering alarms (objective 6). The design in [1] suits the traffic of a network link; applying it to a single host needs hotlists that adapt.

Future work.

1. Run a listener study on the protocol of [1], comparing the base and improved designs on detection time, identification accuracy and false positives.
2. Add the weighted sampling and on/off service model of [1], and restore the paper's attack rates.
3. Capture from a mirrored switch port or a router, and port the helper to Linux and macOS.
4. Test IPv6 capture, a real outage and controlled attack traffic on an isolated network.
5. Learn thresholds from longer baselines instead of setting them by hand, and evaluate the adaptive filter for fatigue over long sessions.
6. Extend the detectors and signatures to other conditions, such as UDP floods and slow scans.

References

[1] L. Axon, J. Happa, M. Goldsmith and S. Creese, "Hearing attacks in network data: An effectiveness study," *Computers & Security*, vol. 83, pp. 367-388, 2019, doi: 10.1016/j.cose.2019.03.004.

[2] L. Axon, J. R. C. Nurse, M. Goldsmith and S. Creese, "A formalised approach to designing sonification systems for network-security monitoring," *Int. J. Advances in Security*, vol. 10, 2017.

[3] L. Axon, B. Alahmadi, J. R. C. Nurse, M. Goldsmith and S. Creese, "Sonification in security operations centres: What do security practitioners think?," arXiv:1807.06706, 2018.

[4] L. Axon, S. Creese, M. Goldsmith and J. R. C. Nurse, "Reflecting on the use of sonification for network monitoring," in *Proc. Int. Conf. Emerging Security Information, Systems and Technologies (SECURWARE)*, IARIA, 2016, pp. 254-261.

[5] M. Gilfix and A. Couch, "Peep (the network auralizer): Monitoring your network with sound," in *Proc. 14th USENIX Large Installation System Administration Conf. (LISA)*, 2000, pp. 109-117.

[6] M. Ballora, N. A. Giacobe and D. L. Hall, "Songs of cyberspace: An update on sonifications of network traffic to support situational awareness," in *Proc. SPIE*, vol. 8064, 2011, Art. no. 80640P.

[7] L. Qi, M. V. Martin, B. Kapralos, M. Green and M. García-Ruiz, "Toward sound-assisted intrusion detection systems," in *On the Move to Meaningful Internet Systems 2007*, Springer, 2007, pp. 1634-1645.

[8] P. Vickers, C. Laing and T. Fairfax, "Sonification of a network's self-organized criticality," arXiv:1407.4705, 2014.

[9] M. Debashi and P. Vickers, "Sonification of network traffic flow for monitoring and situational awareness," *PLoS ONE*, vol. 13, no. 4, Art. no. e0195948, 2018.

[10] M. Debashi and P. Vickers, "Sonification of network traffic for detecting and learning about botnet behavior," *IEEE Access*, vol. 6, 2018.

[11] V. F. Mancuso, E. T. Greenlee, G. Funke, A. Dukes, L. Menke, R. Brown and B. Miller, "Augmenting cyber defender performance and workload through sonified displays," *Procedia Manufacturing*, vol. 3, pp. 5214-5221, 2015.

[12] T. Hermann, A. Hunt and J. G. Neuhoff, Eds., *The Sonification Handbook*. Berlin, Germany: Logos Verlag, 2011.

[13] G. Kramer *et al.*, "Sonification report: Status of the field and research agenda," report prepared for the National Science Foundation by members of the International Community for Auditory Display, 1999.

[14] T. Hildebrandt, T. Hermann and S. Rinderle-Ma, "Continuous sonification enhances adequacy of interactions in peripheral process monitoring," *Int. J. Human-Computer Studies*, vol. 95, pp. 54-65, 2016.

[15] V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, Art. no. 15, 2009.

[16] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Computers & Security*, vol. 28, no. 1-2, pp. 18-28, 2009.

[17] A. de Campo, "A data sonification design space map," in *Proc. 2nd Int. Workshop on Interactive Sonification*, York, UK, 2007.

[18] L. Axon *et al.*, "Sonification to support the monitoring tasks of security operations centres," *IEEE Trans. Dependable and Secure Computing*, 2019, doi: 10.1109/TDSC.2019.2931557.

[19] J. Postel, "Internet Protocol," IETF RFC 791, 1981.

[20] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF RFC 9293, 2022.

[21] W3C, "Web Audio API," W3C Recommendation, 2021. [Online]. Available: https://www.w3.org/TR/webaudio/

[22] WHATWG, "HTML Living Standard: Server-sent events." [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html

[23] Microsoft, "SIO_RCVALL control code," Windows Sockets 2 documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/winsock/sio-rcvall

Appendix A: Running the model

1. Get the files from the repository: https://github.com/RoyceRRNS/Network_Sonifier
2. Open `Network-Sonifier.html` in a browser and press Start listening. Simulated traffic plays at once.
3. Switch an attack on under Inject an attack. Switch the fixes on under Resolved flaws and inject the same attack again to compare.
4. To play a capture, choose Open Wireshark capture and pick a pcap or pcapng file, for example `sample-capture.pcapng` .
5. To play live traffic on Windows, run `python live_capture.py` in the same folder and accept the administrator prompt. The page opens by itself, learns for 30 s and then plays. Close the helper's window to stop.

Appendix B: Source code

The complete source is in the repository named in Appendix A. The files are `Network-Sonifier.html` (the model), `live_capture.py` (the capture helper) and `sample-capture.pcapng` (the test capture). The two extracts below are the core of the mapping and of the anomaly scores, taken from the page with the comments shortened.

```
// One sampled packet becomes three notes: source address, destination address, port.
// A value in its hotlist takes a consonant note by rank; any other takes a dissonant note by hash.
function noteFor(field, value) {
  const rank = (field === 'port' ? hot.ports : hot.ips).get(value);
  if (rank !== undefined) return {midi: CONS[field][rank % CONS[field].length], cons: true};
  return {midi: DIS[field][hash(String(value)) % DIS[field].length], cons: false};
}
const durFor = size => 0.07 + 0.5 * Math.pow(clamp((size - 40) / 1460, 0, 1), 0.8);

// Anomaly scores over the packets sampled in the last 2 s (inside the monitor).
score.ddos = clamp((srcs.size - 5) / 7, 0, 1);      // different new sources opening connections
score.scan = clamp((ports.size - 1) / 5, 0, 1);     // different unusual ports
score.ftp  = clamp((dens - 2) / 4, 0, 1);           // busiest FTP flow
score.any  = Math.max(score.ddos, score.scan, score.ftp,
                      clamp((0.6 - score.link) / 0.35, 0, 1),
                      clamp((share - hot.odd - 0.2) / 0.2, 0, 1));
```